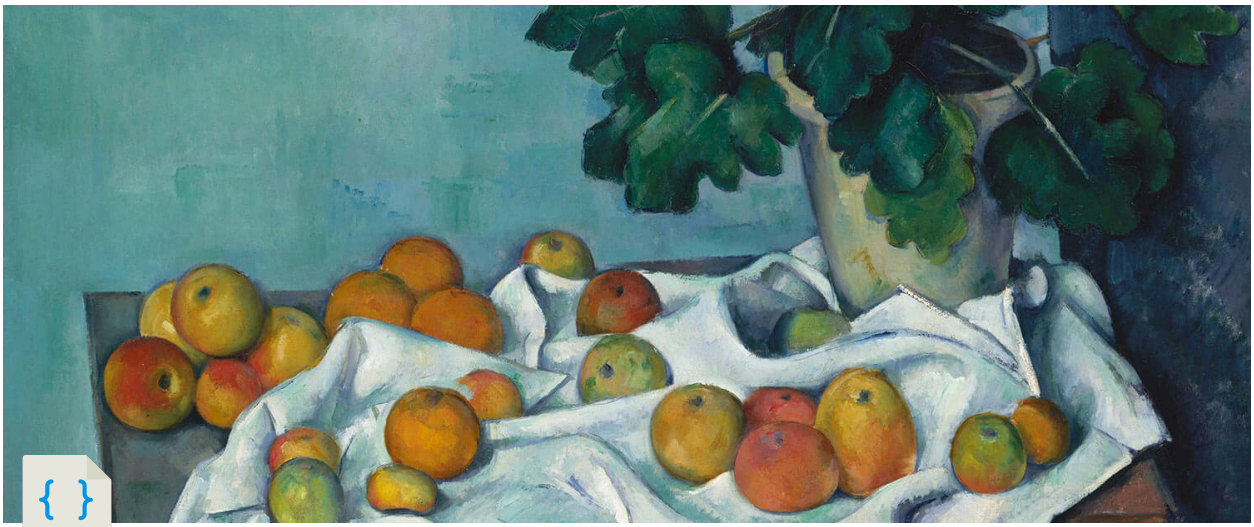


Css {Cascading style sheet}

CSS → cascading style sheet
formatting

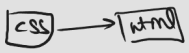
* Implemented using selector *

Types
→ Inline CSS
→ Internal CSS
→ External CSS

1. Internal CSS
 ———
attribute = "value"

2. Internal CSS
html
head
body
<style>
h1 {color: yellow;} → tag selector
</style>

3. External CSS
↳ style.css



* lorem50 → random text for use case
→ class → .name { }
→ id → #name { }
* special selector → Pseudo class Parameter
* ul li:nth-child(3) { background-color: red }
↳ means ul के अन्दर li के 3rd child पर apply हो
* ul li:nth-last-child(2) { background-color: red }
↳ mean ul के अन्दर li के नीचे से second पर apply हो
* ul li:nth-of-type(2n) { background-color: purple }
↳ mean ul के अन्दर li के even number के ऊपर apply हो
* hover
.wrapper:hover { property }

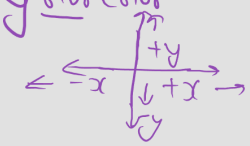
font
text
border → Property

* Font Property

1. font-size: 30px
   ↳ change font size
2. font-family:
   ↳ work on fall back system
   ↳ means font style
3. font-weight: 100
   ↳ ~~कितना~~ कितना size का weight दी उतना bold होगा font
4. font style:
   ↳ italic
   ↳ normal

* Text

1. Text property
   Text-align: center
   Text-decoration: underline, overline {parent need ✗ ✓ 3
   dashed, dotted, wavy {there we make child achieve}
2. Text-decoration-color: black
3. Text-transform: uppercase, lowercase, capitalize
4. Text-shadow: x y blur color



* border

border: 1px solid color
        size

border-radius: 50px {rounder corner border 3

Background
Color
Gradient
→ Property

* background
→ background-color: red;
→ background-image: url(path); // but may can repeat
→ background-repeat: no-repeat
repeat-x

→ background-size: auto // by default
contain // logical scale maintain ratio
cover // cover divor tag size

* color
→ normal → red, green
→ hex
→ rgb
→ hsl

color: white // normal

red green blue alpha value → control opacity // hex color
↳ 0-f
in hexadecimal

rgb(red, green, blue)
↳ 0-255

or

rgba(red, green, blue, alpha value)
↳ 0-255 ↳ 0-1

hsl hue saturation lightness

hsl(h, s, l)
↓ ↳ ↳
0-360 0-100%.
or

hsla(h, s, l, α)

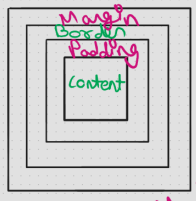
Gradient
→ linear Gradient
→ conic Gradient
→ radial Gradient

linear ← background: linear-gradient(10deg, to right, red, yellow)
background-attachment: fixed
scroll

radial background: radial-gradient(50%, 50%, red, green)

CSS Box Model
Display property
* Box Model

→ Topic Covered



Content → text or tag
Border → boundary of content
Padding → Content or border से अंदर की gap/space
Margin → दिखने में block element है except div उसमें कुछ default margin होता है top & bottom में

Margin
block element
inline
give some error
Top & bottom की value नहीं देते

Value in short → Top right bottom left value
example → border: 1px solid, black
border-width: 1px;
border-style: solid;
border-color: black

Value in short →
padding: Top-bottom left-right
padding: Top left-right bottom
Margin: auto {horizontally align on center}
↳ left, right से & top & bottom से देखते हैं

for height width we do
Sum of content + padding + border + margin
box sizing: border box
↳ used when we have use height, width value and we want fit 300x100px

By content + padding + border + margin की है तो कितना कितना
Property में 300x100 size देंगे content का होगा cssbodel
box की size 300x100 से ज्यादा होगा क्योंकि P+b+m की भी
space होगी। ↳ content size

```
* {
    Margin: 0;
    Padding: 0;
    box-sizing: border-box;
}
```
→ universal selector

* display property

display: block
inline
inline-block
none → make visibility of out call & browser में देखो

Position → Top, left, Right, bottom

Top: 5cm; left: 10cm; → move without margin

static → by default property {T, R, B, L काम नहीं करेगी}

Relative → normal flow में होता है without {T, R, B, L} property work करेगी

absolute → normal flow से बाहर आ जाता है & TRBL will work also.

Fixed → fixed अपनी जगह से चिपक जाता है remaining content scroll होगा

sticky → it scroll up to parent container.

* जब हम किसी में Absolute दे देते तो उसके Parent में Relative दिया

→ like

→ उसके ऊपर एक आ सकता

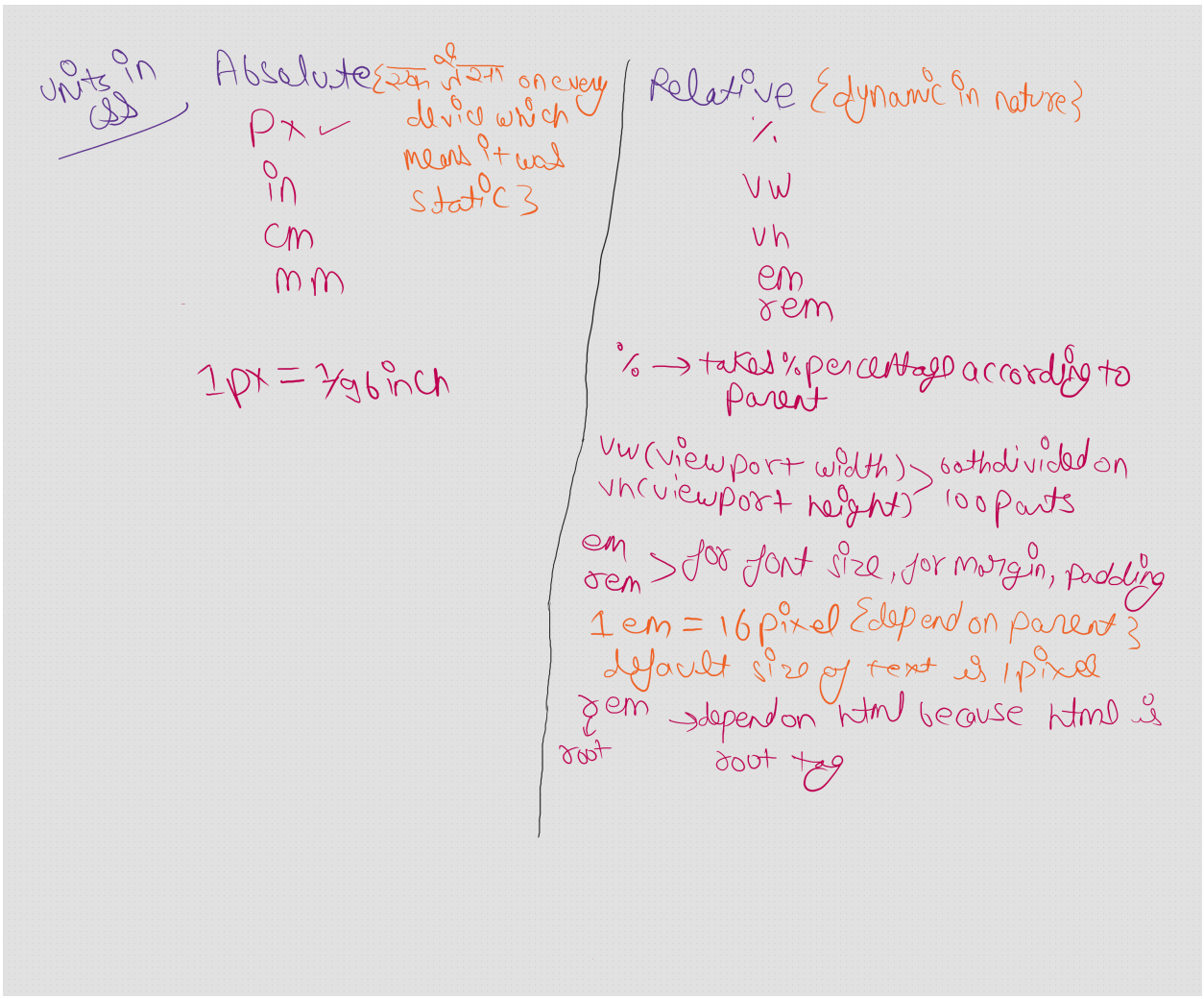
Absolute == fixed, Relative == sticky {Need top value}

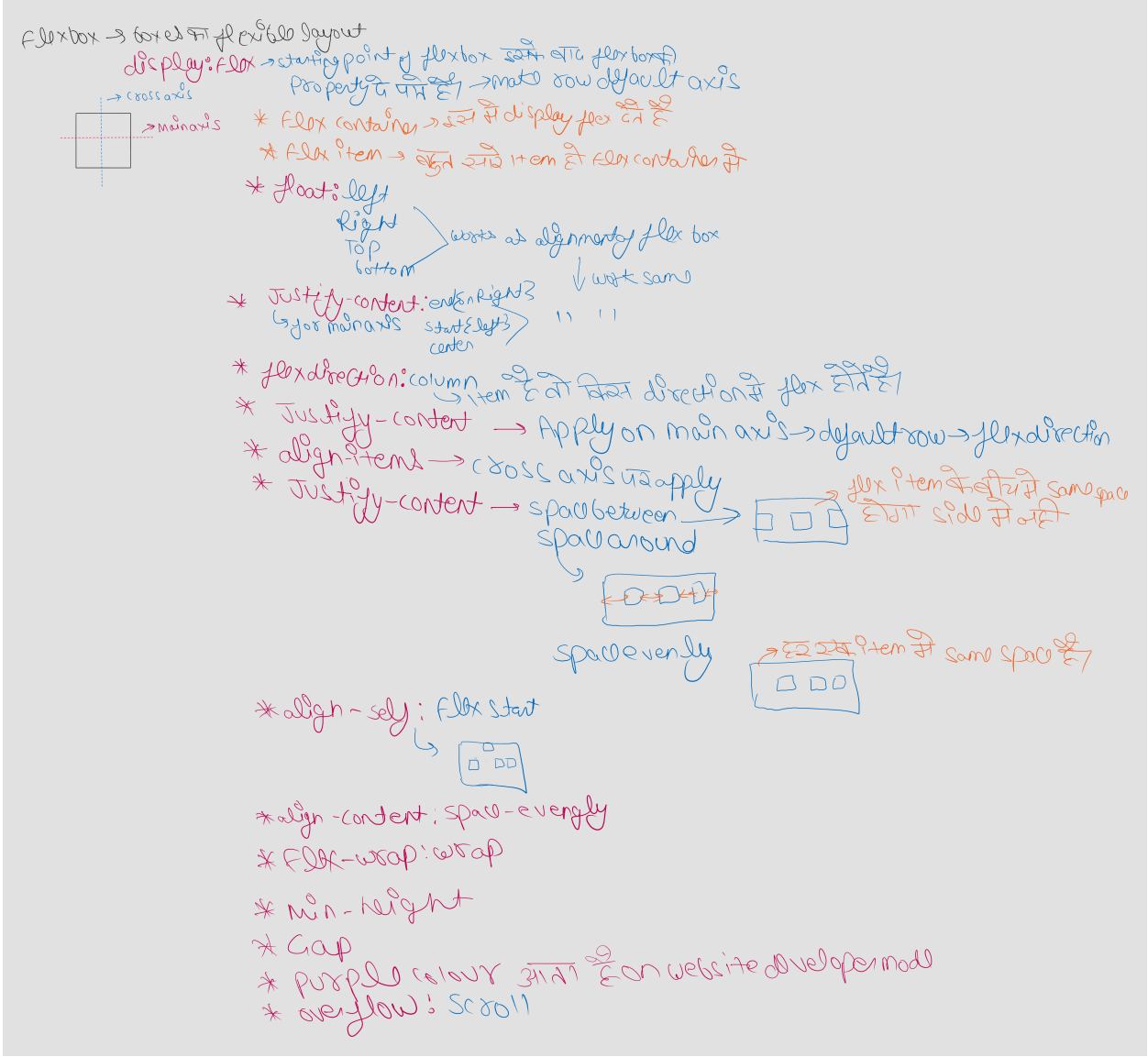
z-index = -1 → जिसकी जितनी ज्यादा z index हो उतना आगे ऊपर उसके ऊपर एक type

* cursor property

cursor: pointer

progress


units in CSS
Absolute {इसका मान on every device which means it was static}
px
in
cm
mm
1px = 1/96 inch
Relative {dynamic in nature}
%
vw
vh
em
rem
% → takes % percentage according to parent
vw (viewport width) > 100 divided on
vh (viewport height) > 100 parts
em
rem > for font size, for margin, padding
1 em = 16 pixel {depend on parent}
default size of text is 1 pixel
rem → depend on html because html is root tag
root


flexbox → box ची flexible layout
display: flex → starting point ची flexbox ला तर flex box ची property दे पण ही → main row default axis
cross axis
main axis
* flex container → ज्याला ही display flex दिले ते
* flex item → सगळे direct item हे flex container चे
* float: left
Right
Top
bottom
works as alignment of flex box
work same
* justify-content: end & Right
for main axis
start & left
center
* flex direction: column → item हे वर वरील direction ने flex होतील
* justify-content → Apply on main axis → default row → flex direction
* align-items → cross axis वर apply
* justify-content → space between → flex item मधील space समान
space around
space evenly → सर्व item मधे same space
* align-self: flex start
* align-content: space-evenly
* Flex-wrap: wrap
* min-height
* Gap
* purple colour आला ह्या on website developer mode
* overflow: scroll

The CSS Flex Container Properties

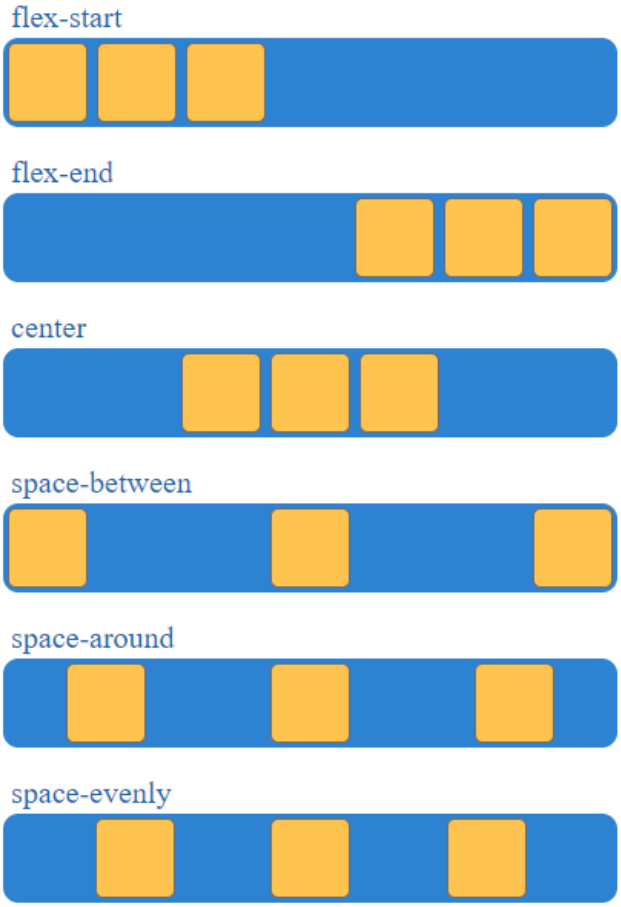
The following table lists all the CSS Flex Container properties:

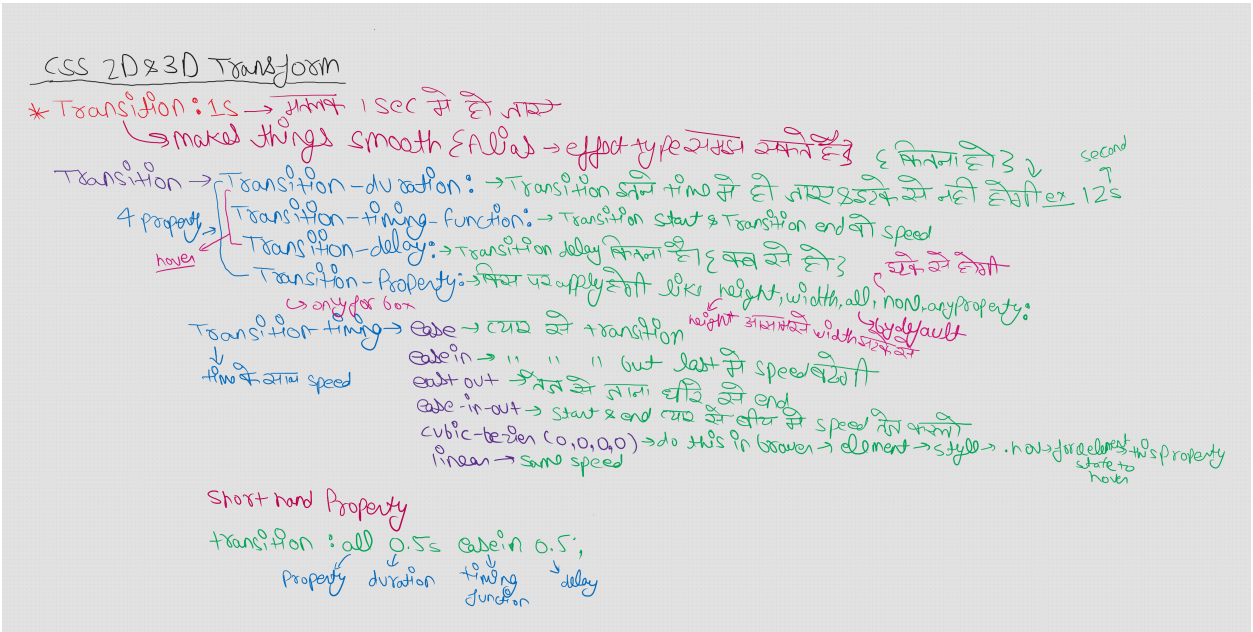
Property	Description
align-content	Modifies the behavior of the flex-wrap property. It is similar to align-items, but instead of aligning flex items, it aligns flex lines
align-items	Vertically aligns the flex items when the items do not use all available space on the cross-axis
display	Specifies the display behavior (the type of rendering box) for an element
flex-direction	Specifies the direction of the flex items inside a flex container
flex-flow	A shorthand property for flex-direction and flex-wrap
flex-wrap	Specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line
justify-content	Horizontally aligns the flex items when the items do not use all available space on the main-axis

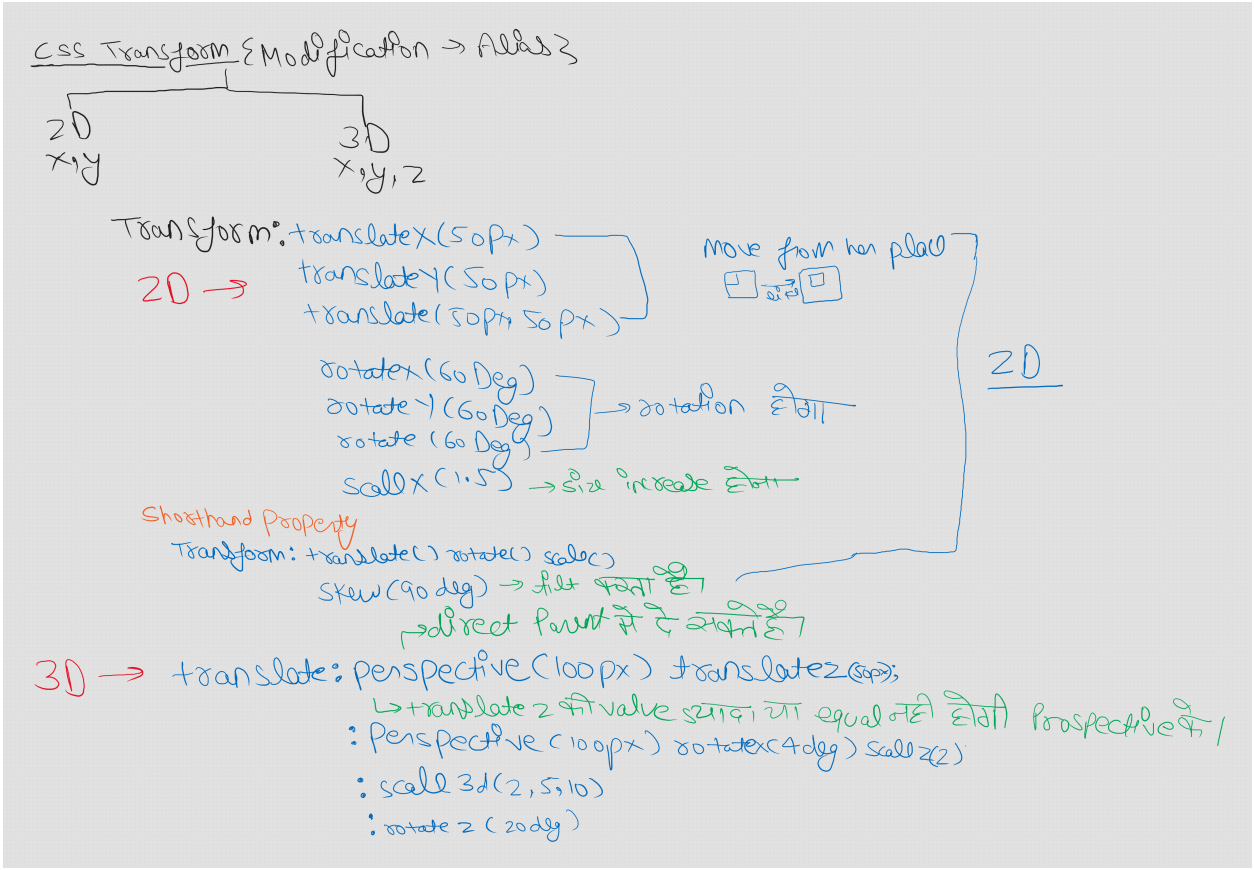
The CSS Flex Items Properties

The following table lists all the CSS Flex Items properties:

Property	Description
align-self	Specifies the alignment for a flex item (overrides the flex container's align-items property)
flex	A shorthand property for the flex-grow, flex-shrink, and the flex-basis properties
flex-basis	Specifies the initial length of a flex item
flex-grow	Specifies how much a flex item will grow relative to the rest of the flex items inside the container
flex-shrink	Specifies how much a flex item will shrink relative to the rest of the flex items inside the container
order	Specifies the order of the flex items inside the container


flex-start
flex-end
center
space-between
space-around
space-evenly


CSS 2D & 3D Transform
* Transition : 1s → 1 sec में होगा
Makes things smooth
Transition → 4 property
Transition-duration :
Transition-timing-function :
Transition-delay :
Transition-Property :
Transition-timing → ease
ease-in
ease-out
ease-in-out
cubic-bezier (0,0,0,0)
linear → same speed
Short hand Property
transition : all 0.5s ease-in 0.5;
property
duration
timing function
delay


Css Transform {Modification → Alias}
2D
x, y
3D
x, y, z
Transform: translateX(50px)
2D →
translateY(50px)
translate(50px, 50px)
Move from her place
rotateX(60Deg)
rotateY(60Deg)
rotate(60Deg)
→ rotation होगा
scaleX(1.5) → size increase होगा
2D
Shorthand Property
Transform: translate() rotate() scale()
skew(90deg) → tilt करता है
↳direct Parent से दे सकते है
3D → translate: perspective(100px) translateZ(50px);
↳translate Z की value ज्यादा या equal नहीं होगी Prospective से।
: perspective(100px) rotateX(4deg) scaleZ(2)
: scale3d(2, 5, 10)
: rotate z (20deg)

Transform: perspective(300px) translate3d(40px,40px,30)
↳ need for 2 axis

Animation :→

transition: is → needs hover
but we need automatic things without hover
Animation → first से last Moment तक का -2 Keyframe बनवाना होते
animation: name duration timing-function delay iteration-count direction fill-mode
→ animation-name: nishant; →
@keyframes nishant {
↳ keyframe को किस Animation पर Apply करना है उसका name आएगा

→ animation-duration → Animation कितने time में complete हो ex 5s
→ animation-timing-function ex 5s: ease in → starting slow last में तेज़
time के साथ speed variation
ease out → start में fast last में slow
linear → constant speed
ease-in-out → start end slow बीच में fast
cubic-bezier(0,0,0,0) → self defined according
→ animation-delay → animation कितने time बाद start होगा
→ animation-iteration count → ex 2,3 → animation कितनी बार चलाना है ex infinite

@keyframes nishant {
from { → initial condition
} to { → final condition
}

→ animation-direction: reverse ← उल्टा से start
alternate आगे → ← आगे
→ animation-fill mode: backwards → starting point पर end होगा
forward → forward end Point पर end होगा
both

@keyframes xyz {
0% {
}
50% {
}
100% {
}
}

!important → It overridden normal css property
ex color: red !important

CSS variables
:root { usually html
--main-color: #ffffff;
--text-color: white;
}
to use it →
.my-box {
background-color: var(--main-color);
color: var(--text-color;
}
variable

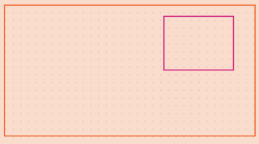
* universal selector ⇒ * { }
* Element selector ⇒ p { }, h1 { }, div { }
* class selector ⇒ .box { }
* id selector ⇒ #header { }
* Group selector ⇒ h1, p, .box { }
* descendant selector ⇒ div p { } → Parent के अन्दर के element को target करेगा (direct + indirect) nested also
* child selector ⇒ ul > li { } → सिर्फ direct children को target करेगा not indirect or nested
* Attribute selector ⇒ input [type="text"] { } → To element specific attribute के साथ Match हो उनको Target करेगा like form, image, link, button etc.
element attribute value
* Pseudo-class ⇒ a : hover { }
selector Pseudo-class
property color: orange; value
}
→ Apply special state to any element

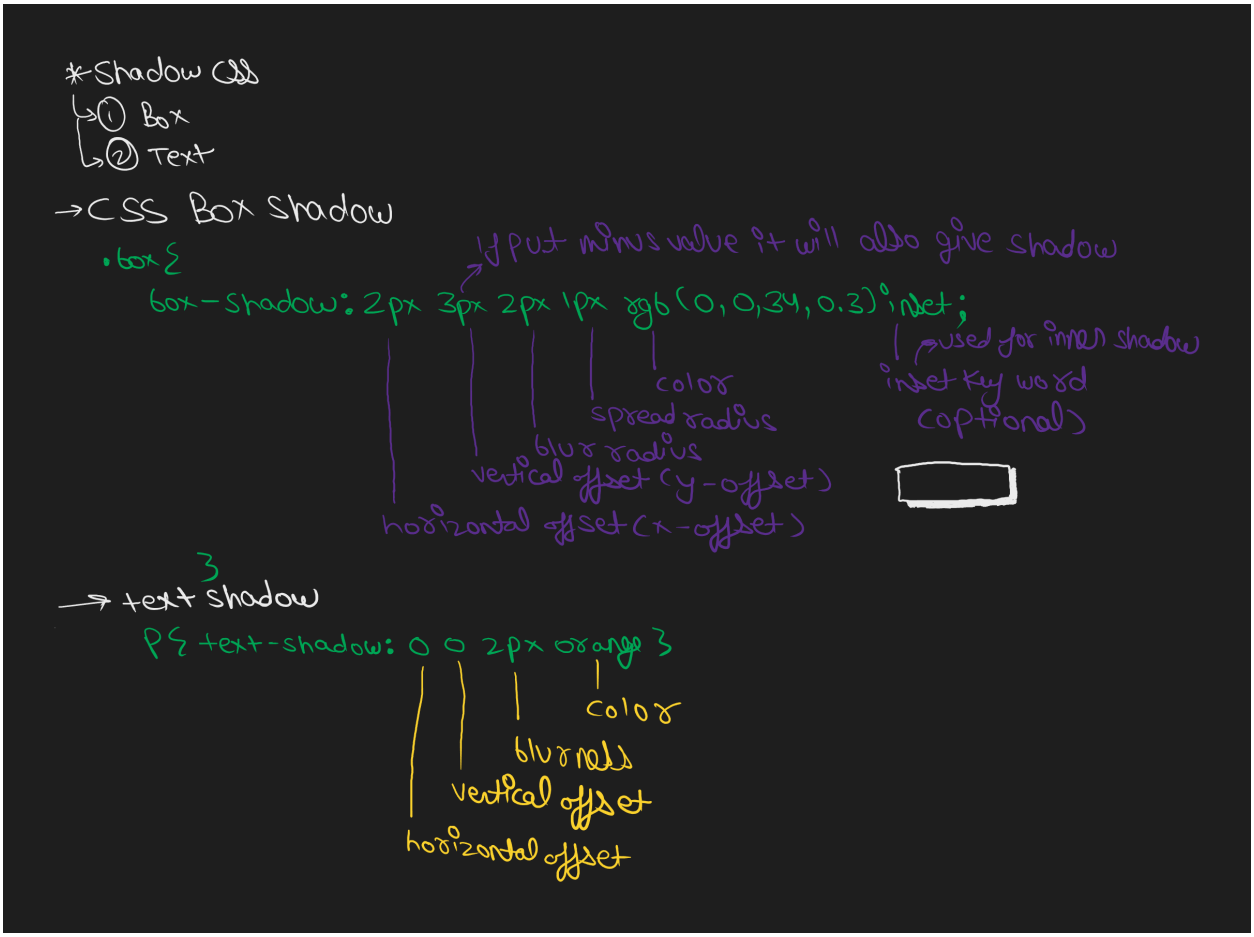
Main Pseudo class
a:link { } → anchor tag में link लगा है तो ये करेंगे
a:hover { } → इसके ऊपर Mouse लेके जाओगे तो
a:visited { } → अगर link खोल लिया है तो apply करो
a:active { } → अगर click करके रखोगे तो

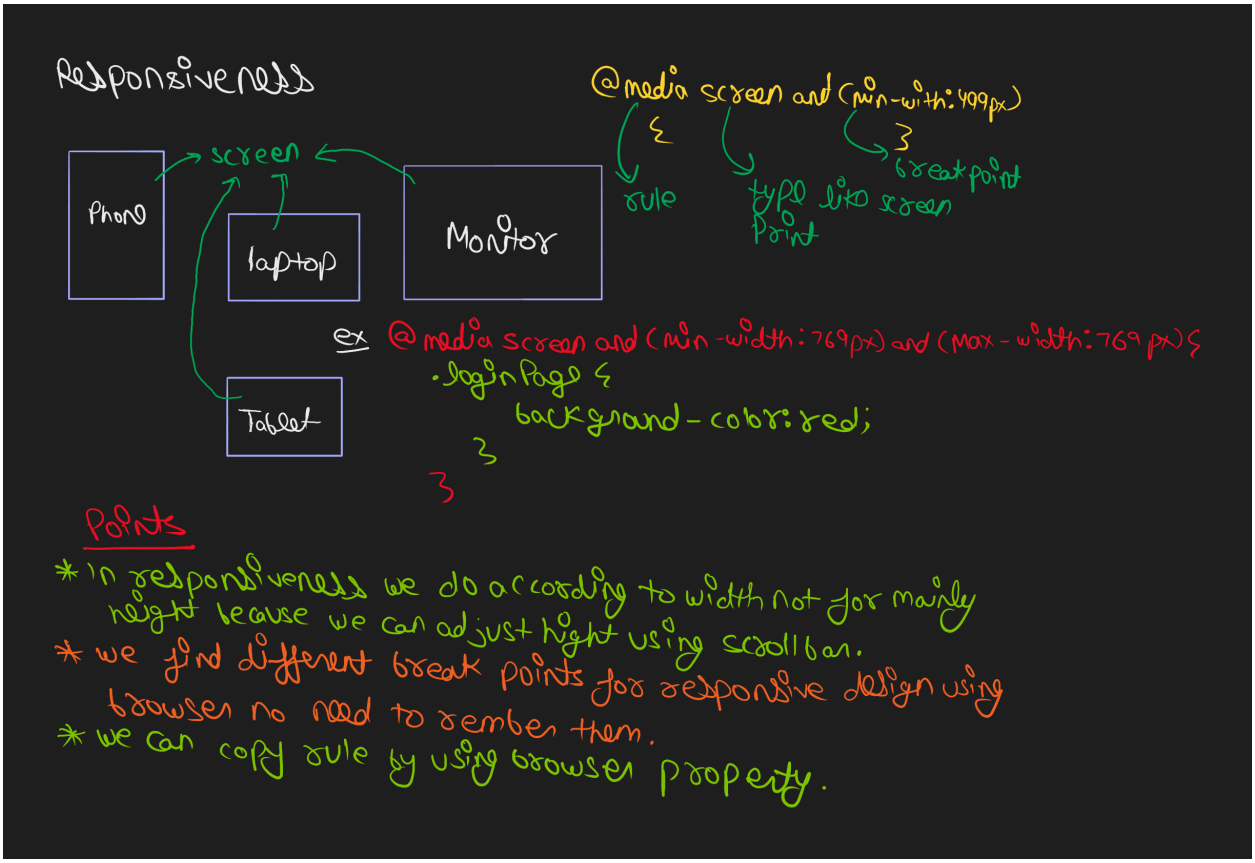
* Pseudo-element selector ⇒ p::selection { } → highlight text selection colour

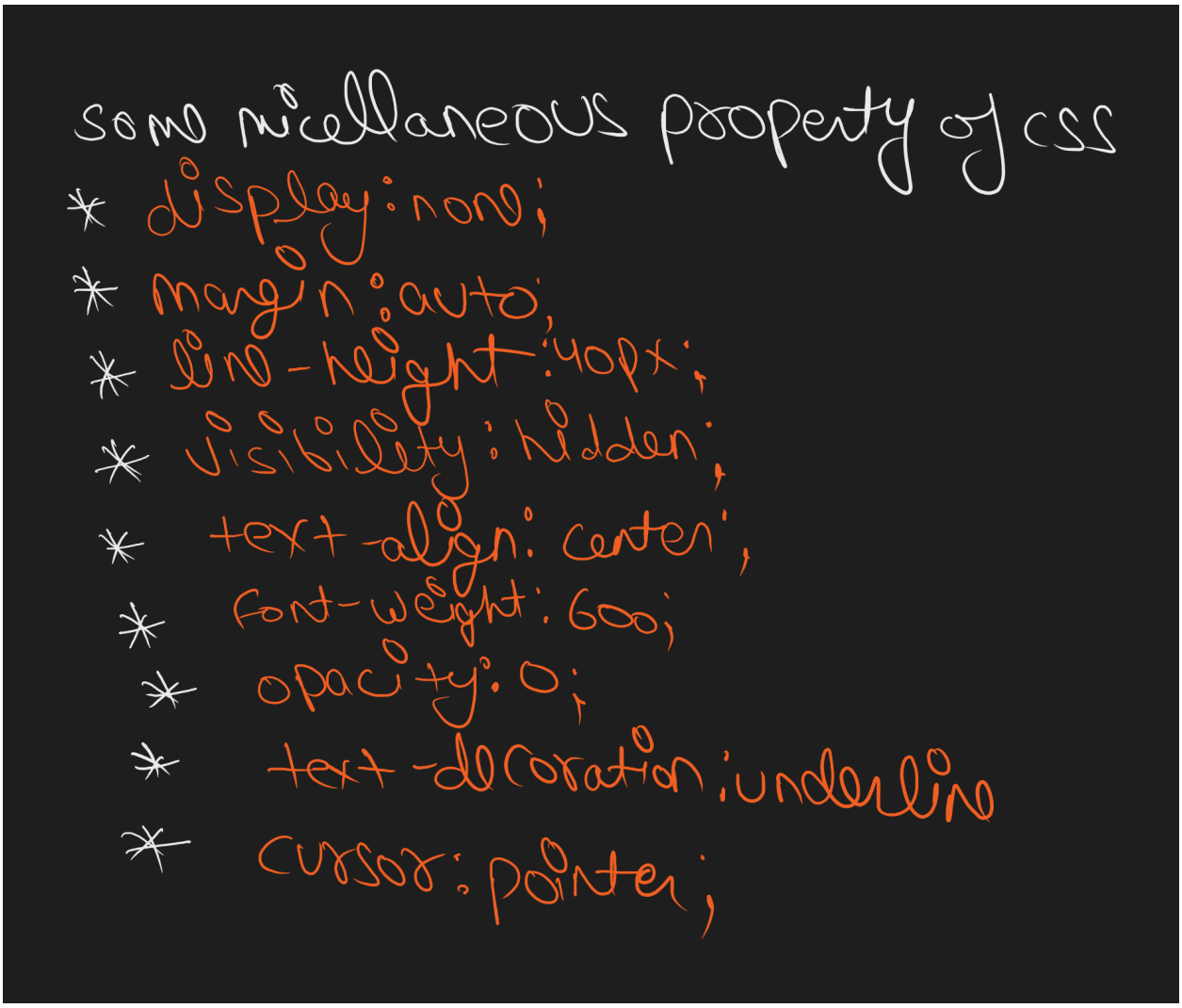
overflow: visible → show overflow data
scroll → show vertical & horizontal scroll bar
hidden → hide overflow data
auto → vertical scrollbar show only not horizontal

Float: none →
left
right
inline-start
inline-end




*Shadow CSS
① Box
② Text
→CSS Box Shadow
.box{
box-shadow: 2px 3px 2px 1px rgb(0,0,34,0.3) inset;
if put minus value it will also give shadow
used for inner shadow
inset key word (optional)
color
spread radius
blur radius
vertical offset (y-offset)
horizontal offset (x-offset)
→ text shadow
p{ text-shadow: 0 0 2px orange;
color
blur radius
vertical offset
horizontal offset


Responsiveness
@media screen and (min-with:499px)
{
}
rule
type like screen
Print
break point
screen
Phone
laptop
Monitor
Tablet
ex @media screen and (min-width:769px) and (max-width:769px){
.loginPage {
background-color:red;
}
}
Points
* in responsiveness we do according to width not for mainly height because we can adjust hight using scrollbar.
* we find different break points for responsive design using browser no need to remember them.
* we can copy rule by using browser property.

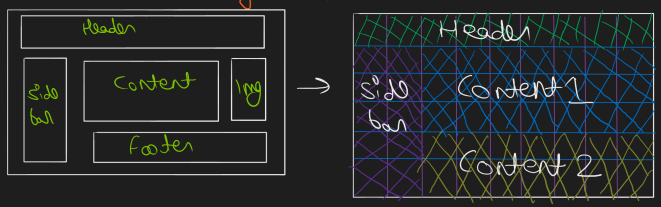

some miscellaneous property of css
* display: none;
* margin: auto;
* line-height: 40px;
* visibility: hidden;
* text-align: center;
* font-weight: 600;
* opacity: 0;
* text-decoration: underline
* cursor: pointer;

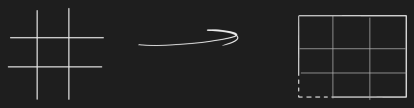
* Grid

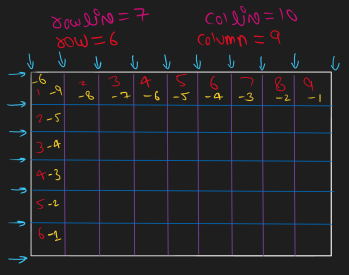
Grid → it is 2 dimention layout { rows & columns }

Flexbox → 2 dimention layout.

always make parent items are display: grid; & inside her items are known as grid items.



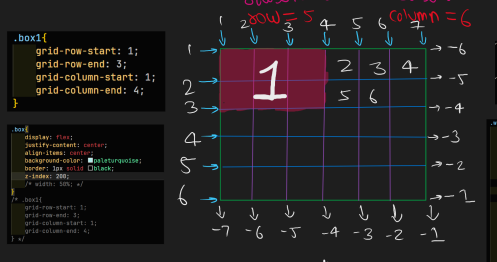




* In browser layout
→ show lineNumber
→ show areaNames
→ Grid overlays
↳ div.wrapper

.parent { display: Grid;
grid template rows: 50px; this & take big size other take all size and size
grid template columns: 100px 300px 2opx 2opx; or repeat(6, 30px); or 1fr 2fr 3fr;
column col2 col3 col4 or (6, 1fr);
fraction means screen size in px / no. of fr

.box { background color: paleturquoise;
/* width: 50px; */




```
.box1{
    grid-row-start: 1;
    grid-row-end: 3;
    grid-column-start: 1;
    grid-column-end: 4;
}
```

```
.box{
    display: flex;
    justify-content: center;
    align-items: center;
    background-color: paleturquoise;
    border: 1px solid black;
    z-index: 200;
    /* width: 50%; */
}
/* .box1{
    grid-row-start: 1;
    grid-row-end: 3;
    grid-column-start: 1;
    grid-column-end: 4;
} */
```

```
.box1{
    grid-area: Header;
}
.box2{
    grid-area: Sidebar;
}
.box3{
    grid-area: Content-1;
}
.box4{
    grid-area: Content-2;
}
```

```
.box6{
    grid-area: 1/ 1/ 5 /3;
    grid-row: 1/5;
    grid-column: 4/6;
    z-index: 1;
}
```

row start, col start, row end, col end

```
.wrapper {
    display: grid;
    width: 60%;
    height: 50%;
    background: red;
    grid-template-columns: repeat(3, 1fr);
    grid-template-rows: repeat(3, 1fr);
    /* grid-template-rows: repeat(auto-fill,
    minmax(30px, 1fr)); */
    /* grid-template-columns: 100px 200px
    100px; */
    /* grid-template-columns: 1fr 9fr 1fr; */
    /* grid-template-rows: repeat(2,1fr); */
    /* grid-gap: 20px; */
    gap: 50px;
    grid-template-areas:
      "  Header Header Header"
      "Sidebar Content-1"
      "Sidebar Content-2";
}
```

Grid container

- → gap
- → grid template columns
- → grid template rows
- → Justify content
- → align-content
- → align-items

Grid child items

- → justify-self
- → align-self
- → grid-column
- → grid-row
- → grid-area

Grid template area

→ used for name base grid defining

```
.grid-container{
grid-template-area:
'header header header header header header'
'menu main main main right right'
'menu footer footer footer footer footer';
}
.item1{grid-area: header;}
.items2{grid-area: menu;}
.item3{grid-area: main;}
.item4{grid-area: right;}
.item5{grid-area:footer;}
```